



CPU DAI driver



CPU DAI driver

- ▶ The CPU DAI driver is now a component driver, like the codec ones.
- ▶ However, it is usually more complex as it need to handle IRQs and take care of pinmuxing, clocks and DMA.
- ▶ Also, the list of supported format and rates is usually very large.



DMA handling

- ▶ When a DMA controller is available, handling DMA in ALSA is done almost completely in the core, through `dmaengine_pcm`.
- ▶ The DMA is simply registered using `devm_snd_dmaengine_pcm_register()` . This handles parsing the device tree if necessary.
- ▶ In the DAI driver probe callback, the DMA engine is simply configured using `snd_soc_dai_init_dma_data()` which takes the DMA configuration for playback and capture.



DMA handling example

sound/soc/atmel/atmel-i2s.c

```
struct atmel_i2s_dev {
    struct device          *dev;
    struct regmap          *regmap;
    struct clk             *pclk;
    struct clk             *gclk;
    struct snd_dmaengine_dai_dma_data playback;
    struct snd_dmaengine_dai_dma_data capture;
    unsigned int           fmt;
    const struct atmel_i2s_gck_param *gck_param;
    const struct atmel_i2s_caps *caps;
    int                    clk_use_no;
};
[...]
static int atmel_i2s_dai_probe(struct snd_soc_dai *dai)
{
    struct atmel_i2s_dev *dev = snd_soc_dai_get_drvdata(dai);

    snd_soc_dai_init_dma_data(dai, &dev->playback, &dev->capture);
    return 0;
}
```



DMA handling example

sound/soc/atmel/atmel-i2s.c

```
static int atmel_i2s_probe(struct platform_device *pdev)
{
    [...]

    /* Prepare DMA config. */
    dev->playback.addr      = (dma_addr_t)mem->start + ATMEL_I2SC_THR;
    dev->playback.maxburst   = 1;
    dev->capture.addr        = (dma_addr_t)mem->start + ATMEL_I2SC_RHR;
    dev->capture.maxburst    = 1;

    if (of_property_match_string(np, "dma-names", "rx-tx") == 0)
        pcm_flags |= SND_DMAENGINE_PCM_FLAG_HALF_DUPLEX;
    err = devm_snd_dmaengine_pcm_register(&pdev->dev, NULL, pcm_flags);
    if (err) {
        dev_err(&pdev->dev, "failed to register PCM: %dn", err);
        clk_disable_unprepare(dev->pclk);
        return err;
    }

    [...]
}
```



DMA handling

- ▶ When a peripheral DMA controller is used, this is more complex.
- ▶ The driver will have to handle all the aspects of the PCM stream life cycle.
- ▶ Understandable example in [sound/soc/atmel/atmel-pcm-pdc.c](https://github.com/linux-sound/atmel-pcm-pdc.c)